

A Modern Ecommerce System Design

Bounded contexts, aggregates, commands, and context relationships (sequence & architecture views)

The system is decomposed into nine bounded contexts, grouped under four teams (Catalog & Discovery, Checkout, Supply, Consumer, see Section 4). Checkout is orchestrated by a process manager so that stock reservation, payment authorization, and capture stay strictly ordered. Everything downstream (fulfillment, notifications, search projections) is choreographed through domain events. Each context is deployed as its own service (Section 4), so contexts under heavier load can scale independently of the rest of the system.

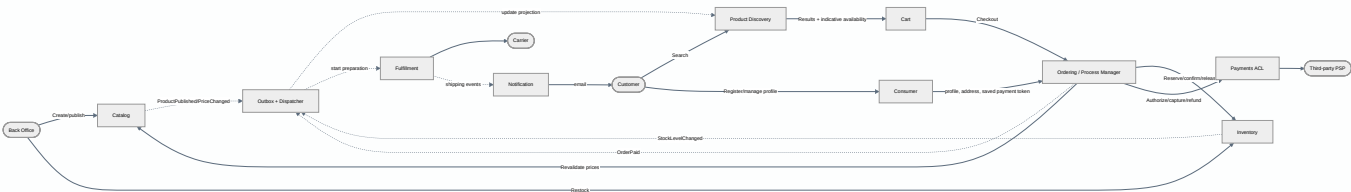
1 Bounded Contexts

Table 1: Bounded contexts and their responsibilities

Context	Responsible for	Not responsible for
Catalog	Product master data, descriptions, media, list prices, publishing	Available quantities
Inventory	Stock levels per product, reservations, restocking, non-overselling invariant	Prices, orders, payments, shipping
Cart	Customer/guest cart, cart lines, and price snapshots	Guaranteeing availability at checkout
Consumer	Registered user profile, saved addresses, saved payment method tokens, order history view	Auth credentials (external IdP), guest checkout data, cart contents
Ordering / Checkout	Orders, state machine, checkout orchestration, and compensations	Payment details or the physical handling of stock and shipping
Payments	Abstraction over the payment service, authorization, capture, void/refund, and reconciliation	Stock and availability
Fulfillment	Preparation and shipping of paid orders, tracking	Payments, pricing, and commercial availability
Product Discovery	Full-text index on product names and descriptions, filters, indicative availability	Any domain write operation
Notification	Transactional emails and order/shipping status notifications	Physical order preparation or shipping

Figure 1: Context relationships and event flow

Solid = synchronous call. Dashed = asynchronous domain event via outbox.



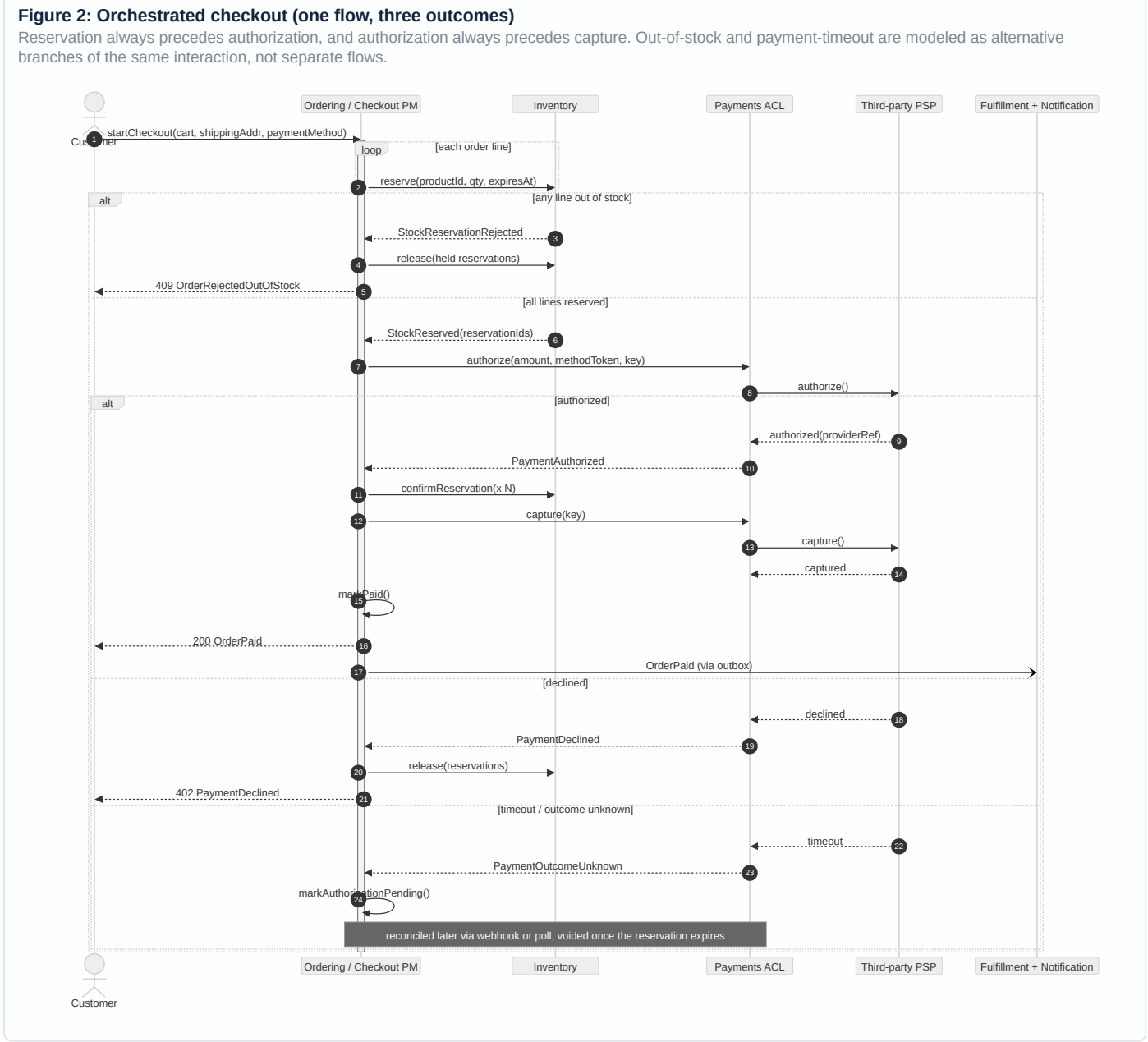
2 Aggregates, Entities, and Commands

Table 2: Aggregate roots, key commands and invariants

Aggregate	Commands → emitted events	Key invariant
Product Catalog	create, attachMedia, updateDetails, changePrice, publish, discontinue → Product{Created,DetailsUpdated,Published,Discontinued}, PriceChanged	A published product has a name, description, price, and at least 1 image
StockItem Inventory	receiveStock, reserve, confirmReservation, releaseReservation, reverseConfirmation, adjustStock → Stock{Received,Reserved,Confirmed,Released,ConfirmationReversed} + StockLevelChanged	available = onHand - reserved ≥ 0
Cart Cart	addItem, changeQuantity, removeItem, mergeIntoCustomerCart → CartUpdated, CartMerged	Positive quantities, one line per product, not expired or already in checkout
Consumer Consumer	register, updateProfile, addAddress, savePaymentMethod → ConsumerRegistered, ProfileUpdated, AddressAdded, PaymentMethodSaved	Email unique across consumers, exactly one default shipping address at a time
Order Ordering	startCheckout, confirmAllStockReserved, markPaymentAuthorized, confirmStock, markPaid, rejectOutOfStock, cancel → CheckoutStarted ... OrderPaid / OrderRejectedOutOfStock / OrderCancelled	No payment without an active reservation per line. Paid requires confirmed stock and captured payment. Totals frozen once authorization starts

Payment Payments	authorize, capture, voidAuthorization, refund → PaymentAuthorized/Declined/OutcomeUnknown, Captured/CaptureFailed, AuthorizationVoided, Refunded	Retries never duplicate a charge (idempotency key). Active captures ≤ order total
Shipment Fulfillment	prepare, markShipped → ShipmentPrepared, OrderShipped	Created only for a paid order. Cannot ship without tracking info

3 Checkout Sequence: Happy Path, Out-of-Stock, Payment Timeout



4 Physical Architecture

Table 3: Monolith vs. microservices

Option	Pros	Cons
Modular monolith	Fast to change: one codebase, no cross-service coordination. Zero network calls between modules, everything runs in-process. A single well-sized machine (plus a few replicas) goes a long way early on.	The whole application scales as one unit. A hot context (search, cart) drags the rest of the process up with it.
Microservices	Each service scales on its own, tuned to its own load, a clear win as traffic grows. Plays well with a growing org: teams ship and deploy on their own schedule.	Harder to maintain: real complexity from events and HTTP calls between services, and failures now cross process boundaries.

Decision: microservices, chosen for the scalability of the system as a whole. Traffic is uneven across contexts (Product Discovery and Cart see far more load than Fulfillment or Notification), so independent scaling matters more than the simplicity of one process. Bounded contexts already share no aggregates and talk only through explicit commands and events, so moving from in-process calls to HTTP and a message broker is a transport change, not a redesign.

Figure 3: Microservices, one service per bounded context

Solid = synchronous HTTP call. Dashed = asynchronous domain event via a message broker. Each service owns its schema and database (not shown for brevity).

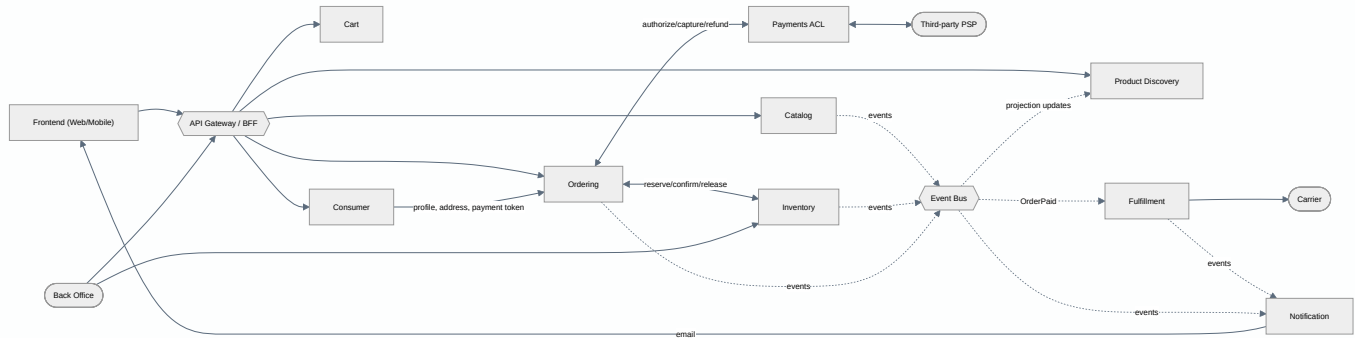


Table 4: Team ownership

Team	Owned services
Catalog & Discovery	Catalog, Product Discovery
Checkout	Cart, Ordering, Payments
Supply	Inventory, Fulfillment
Consumer	Consumer service, Frontend/BFF, Notification

Services share no aggregates and no database. Cross-service calls go through the gateway or the event bus only, never straight to another service's schema.

5 Edge Cases and Requirement Coverage

- **Partial reservation failure:** already-held reservations are released before responding. There is no partially-valid multi-product hold. Reservation TTL is the safety net if compensation itself fails to run.
- **Ambiguous payment outcome:** a timeout is not a decline. Webhook and polling reconcile via `ProviderReference / IdempotencyKey`. A late authorization is voided, never captured, if the reservation already expired.
- **Capture/confirmation failure:** authorization is voided and confirmed reservations are compensated via `reverseConfirmation`, both idempotent and audit-logged.
- **Price drift:** prices are revalidated between add-to-cart and checkout. Totals become immutable once authorization starts.

Table 5: Requirement coverage

Requirement	Mechanism
Search by name/description, add to cart if available	Search projection (Fig. 1), best-effort <code>inStock</code> check, authoritative reservation at checkout (Fig. 2)
Back office: product + image/description/qty, restock	<code>Product</code> and <code>StockItem.receiveStock()</code> (Table 2)
Checkout with address + payment method, guest allowed	<code>Order.Buyer</code> : required email, optional <code>customerId</code>
Payments via third party, stock kept independent of payments	<code>Payments</code> ACL never reads stock, <code>Inventory</code> never calls the PSP
Out of stock blocks payment and is shown in the UI	Payment is only ever called after all reservations succeed (Fig. 2, branch 1)
Order preparation, shipping, and notifications	<code>Fulfillment</code> + <code>Notification</code> , event-driven off <code>OrderPaid</code> (Fig. 1)

Impact. The design protects one thing above all: no customer is charged before stock is reserved and confirmed, and no unit is sold twice.

Quality. Scalability: storefront reads are served by a projection that never touches aggregates with invariants, and each service scales on its own hardware, so a hot context (search, cart) never drags the rest of the system up with it. Resilience: each service's outbox and inbox prevent dual writes, idempotency and reconciliation make retries and webhooks safe across service boundaries, and TTLs plus compensations protect stock. Responsiveness: projections, notifications, and fulfillment run asynchronously, and the process manager keeps checkout progress observable even when the PSP responds slowly. Accepted costs: stock stays held temporarily for abandoned checkouts, storefront availability can be briefly stale (corrected at checkout), and cross-service calls and events add real operational complexity in exchange for independent scaling. Monitoring must cover expired reservations, stuck processes, unknown PSP outcomes, outbox backlog, and inter-service latency.

Effort. Each service maps 1:1 to a team-owned bounded context, so ownership is unambiguous from day one. The added operational cost, deploys, observability, and contracts between services, is accepted up front in exchange for scaling independently as traffic and headcount grow.